EXPERIMENT – 8

Title: Implementation of Selective Repeat Mechanism

Objective:

To understand data link layer flow control mechanism under noisy channel.

Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

Selective Repeat ARQ is a flow control and error control protocol used in the Data Link Layer for reliable data transmission over noisy channels. It is a type of sliding window protocol where multiple frames can be sent before receiving acknowledgements.

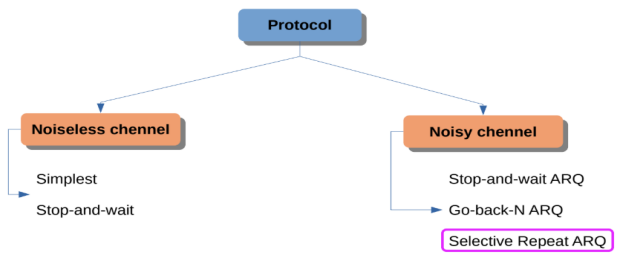
In this mechanism, the sender transmits several frames continuously within a fixed window size. Each frame contains a sequence number so that the receiver can identify them correctly. The receiver checks every incoming frame and sends ACK for correctly received frames and NACK for lost or damaged frames.

Unlike Go-Back-N ARQ, Selective Repeat retransmits only the specific lost or corrupted frame instead of retransmitting all frames after it. This improves bandwidth efficiency and reduces unnecessary retransmission.

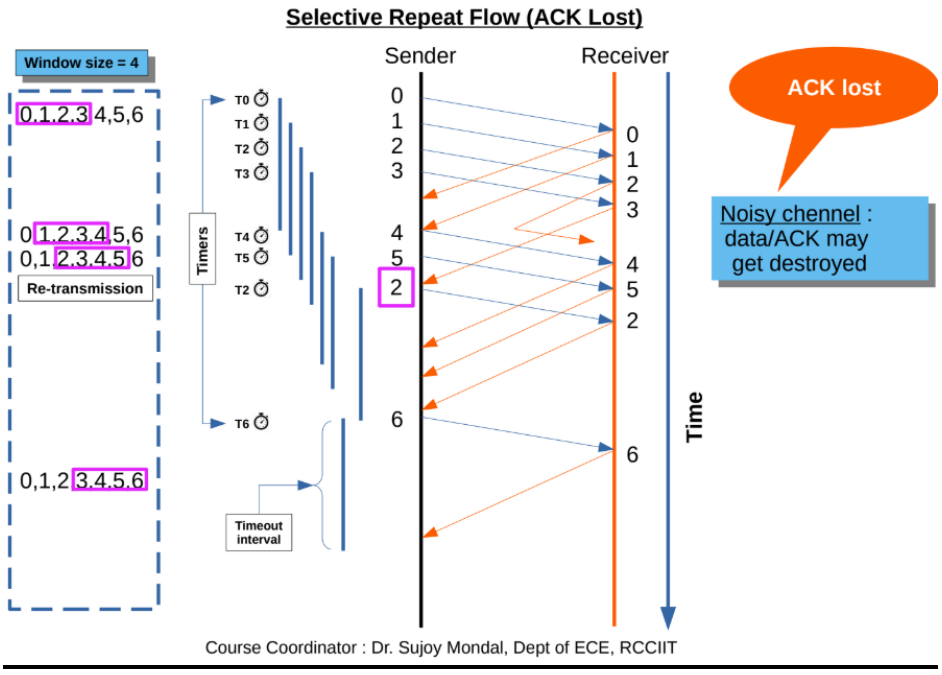
Both sender and receiver maintain separate windows. The sender stores transmitted frames temporarily until ACK is received. If an ACK is not received within the timeout period, the sender retransmits that particular frame.

Selective Repeat ARQ is mainly used in noisy communication channels where frame loss or ACK loss may occur. It provides better performance and efficient communication compared to Stop-and-Wait and Go-Back-N protocols.

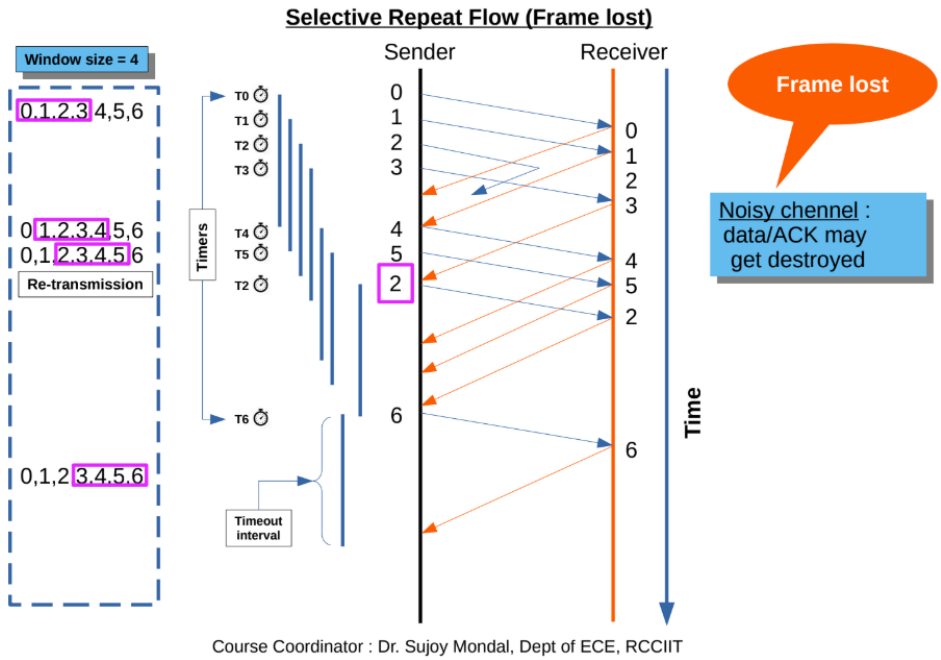
Protocol Concept:



Selective Repeat Flow (ACK Lost):



Selective Repeat Flow (Frame lost):



Procedure / Algorithm

Sender Side

1. Create UDP server socket.
2. Define sender window size.

3. Send frames one by one.

4. Start timer for each frame.

5. Wait for ACK/NACK.
6. Retransmit frame if timeout occurs.
7. Stop after all frames are acknowledged.

Receiver Side

1. Create UDP client socket.

2. Receive frames from sender.

3. Check frame condition.

4. Send ACK if frame received correctly.

5. Send NACK if frame lost.

6. Store received data in buffer.

7. Display final received message.

Program Code

Sender Program:

```python
import time
import selective_repeat_udp_module as udp
import netpack as npk

# Parameters:
m=3
N=int((2**m)/2)
SF=0
SL=SF+N-1
S=SF
frame_timeout_val=5
print('Window size is considered as : {}'.format(N))

# Data:
data_string='RCCIIT ECE@SMN'
data=[val for val in data_string]
# print(data)

# timer & ack_frame:
timer=[]
ack_frame=[]

# ========================================================
# New frame Tx method:
# ========================================================
def tx_frame(server_socket, addr):
    global data, timer, ack_frame, S


    ack_frame.append(0)
    # SEND data[S] via socket:
    msg=data[S]+'-'+str(S)
    npk.send_data_to_udp_client(server_socket, msg, addr)
    timer.append(time.time())
    S+=1
    print('SENT : {}'.format(msg))

# ========================================================
# Re-Tx frame method:
# ========================================================
def retx_frame(i,server_socket, addr):
```

```python
# ========================================================
def retx_frame(i,server_socket, addr):
    global data, timer, ack_frame
    # global socket, addr

    ack_frame[i]=0
    # SEND data[i] via socket:
    msg=data[i]+'-'+str(i)
    npk.send_data_to_udp_client(server_socket, msg, addr)
    timer[i]=time.time()
    print('[Re-Tx] frame : {}'.format(i))

# ========================================================
# Main section:
# ========================================================
if __name__=='__main__':

    # START SERVER:
    server, ADDR, addr = udp.server_method()
    time.sleep(5)

    while True:

        if S==len(data) and sum(ack_frame)==len(data):
            # CLOSE SOCKET:
            npk.send_data_to_udp_client(server, "QUIT", addr)
            npk.close_socket_connection(server)
            print('SENDER MESSAGE : {}'.format(data_string))
            break

        # ---------------------------------
        # TX new frame section:
        if S<=SL:
            tx_frame(server, addr)
            time.sleep(1) # To creat a delay of atlest 1 sec be


        # Look at ACK Rx: check received msg
        # msg format : ACK-X / NACK-X
        # msg='ACK-10' means frame 10 is good and received cor
        # msg='NACK-10' means frame 10 having trouble need ret
```

```python
        # ----------------------------------
        try:
            (msg, addr)=npk.read_data_from_udp_client(server)

        except:
            pass
        else:
            msg_split=msg.split('-')
            x=int(msg_split[-1])
            print('RECV : {}'.format(msg_split))

            if msg_split[0]=='ACK':
                # for i in range(SF,S):
                #     ack_frame[i]=1
                #     timer[i]=-1
                ack_frame[x]=1
                timer[x]=-1

                # update window position:
                for id in range(SF,len(ack_frame)):
                    if ack_frame[id]==1:
                        SF=id+1
                        SL=min(SF+N-1,len(data)-1) # Must not exceed last
                    elif ack_frame[id]==0:
                        # print('S:{},SF:{},SL:{}'.format(S,SF,SL))
                        break
            elif msg_split[0]=='NACK':
                retx_frame(x,server, addr)

        finally:
            pass

        # ----------------------------------
        # Look at timer:
        T=time.time()
        # print('Timer Checking')
        for i in range(SF,S):
            if ack_frame[i]==0 and round((T-timer[i])>frame_timeout_val):
                print('[TIMER EXPIRES FOR FRAME : {}]'.format(i))
                retx_frame(i,server, addr)
```

Receiver Program:

```python
import random
import time
import selective_repeat_udp_module as udp
import netpack as npk

# Parameters:
m=3
N=int((2**m)/2)
R=0
count=0
print('Window size is considered as : {}'.format(N))

# Data:
data=[]
buffer=[0 for i in range(N)]

#=========================================================
# Generate rando flag: 0-Normal, 1-Frame lost, 2-ACK lost
#=========================================================
def generateFlag():
    val=random.randint(0,100)
    if val<30 and val>=0:
        return 0
    elif val<80 and val>=30:
        return 1
    else:
        return 2

#=========================================================
# Wait random time : Multiple of 100 ms
# min 0 sec & max 1 sec
#=========================================================
def wait():
    cnt=random.randint(0,10)
    for _ in range(cnt):
        time.sleep(0.1)

#=========================================================
# Send ACK:
#=========================================================
def send_ack(i, client_socket, ADDR):
    # SEND MSG='ACK-'+str(i) to Socket
    msg='ACK-'+str(i)
    npk.send_data_to_udp_server(client_socket, msg, ADDR)
    print('SENT : {}'.format(msg))

#=========================================================
# Send nACK:
#=========================================================
def send_nack(i,client_socket, ADDR):
    # SEND MSG='NACK-'+str(i) to Socket
```

```python
def send_nack(i,client_socket, ADDR):
    # SEND MSG='NACK-'+str(i) to Socket
    msg='NACK-'+str(i)
    npk.send_data_to_udp_server(client_socket, msg, ADDR)
    print('SENT : {}'.format(msg))

# =========================================================
# Store received msg to data list:
# =========================================================
def storedata(value,i, client_socket, ADDR): # this is called only when valid frames are received
    global R, N

    # Modified part
    if R==i:
        data.append(value)
        R+=1
    elif R<i: # Frame gap means frame lost
        lost_frame_cnt=i-R # lost frame counts
        for id in range(lost_frame_cnt):
            send_nack(R+id, client_socket, ADDR)
            data.append('')
        data.append(value)
        R=i+1
    else:
        data[i]=value

    print('[=] Received buffer : {}'.format(data))

# =========================================================
# Printing received data:
# =========================================================
def print_data():
    global data
    data_string=''
    print(data)
    for i in data:
        data_string+=i
    return data_string

# =========================================================
# Main section:
# =========================================================
if __name__=='__main__':

    # START CLIENT:
    client, ADDR = udp.client_method()
    time.sleep(5)

    while True:
```

```python
    # START CLIENT:
    client, ADDR = udp.client_method()
    time.sleep(5)

    while True:

        # -------------------------
        try:
            (msg, ADDR)=npk.read_data_from_udp_server(client)
        except:
            pass
        else:

            # CLOSE SOCKET
            if msg=='QUIT':
                client.close()
                print('RECEIVED MESSAGE : {}'.format
                (print_data()))
                break

            msg_split=msg.split('-')
            print('-'*50)
            print('RECV : {}'.format(msg_split))
            x=int(msg_split[-1])
            val=msg_split[0]

            # -------------------------
            flag=generateFlag()
            # flag=0
            # -------------------------

            # -------------------------
            if flag==0: # Normal case : ACK
                print('RECV OK')
                storedata(val,x, client, ADDR)
                send_ack(x, client, ADDR)
                wait()
            elif flag==2: # ACK lost
                print('ACK LOST')
                storedata(val,x, client, ADDR)
            else: # Frame lost case : NACK
                print('FRAME LOST')

        finally:
            pass
```

Selective Repeat Module

```python
[This module used to setup UDP Server & Client sockets]
-------------------------------------------------------
Methods:
[1] server_method()
[2] client_method()
'''
import netpack as npk

# Global Parameters:
SERVER_PORT=9090
SERVER_IP="127.0.0.1"
ADDR=(SERVER_IP, SERVER_PORT)
timeout_val=0.1

# ========================================================
# Server Connection setup
# ========================================================
def server_method():
    '''
    -------------------------
    server_method() returns -
    -------------------------
    [1] Server socket
    [2] ADDR : (server_ip, server_port)
    [3] addr : (client_ip, client_port)
    '''
    global ADDR, timeout_val

    server_socket=npk.create_udp_timeoutsocket(timeout_val)

    npk.start_udp_server(server_socket, ADDR)
    print('[=] UDP Server is starting at : {}'.format(ADDR))

    print('-'*50)
    print('[=] Server waits for connection ...')

    while True:
        try:
            (msg, addr)=npk.read_data_from_udp_client(server_socket)
        except:
            pass
        else:
            npk.send_data_to_udp_client(server_socket, "OK", addr)
            print('[CONNECTED] Client {}'.format(addr))
            break
        finally:
            pass

    return server_socket, ADDR, addr
```

```python
    print('[=] Server waits for connection ...')

    while True:
        try:
            (msg, addr)=npk.read_data_from_udp_client(server_socket)
        except:
            pass
        else:
            npk.send_data_to_udp_client(server_socket, "OK", addr)
            print('[CONNECTED] Client {}'.format(addr))
            break
        finally:
            pass

    return server_socket, ADDR, addr


# ========================================================
# Client Connection setup
# ========================================================
def client_method():
    '''
    -------------------------
    client_method() returns -
    -------------------------
    [1] Client socket
    [2] ADDR : (server_ip, server_port)
    '''
    global ADDR, timeout_val

    client_socket=npk.create_udp_timeoutsocket(timeout_val)

    npk.send_data_to_udp_server(client_socket, "HELLO", ADDR)
    # print('[=] Client is connecting to Server {}'.format(ADDR))

    while True:
        try:
            (msg, ADDR) = npk.read_data_from_udp_server(client_socket)
        except:
            pass
        else:
            if msg=='OK':
                print('[CONNECTED] Server : {}'.format(ADDR))
                break
        finally:
            pass

    return client_socket, ADDR
```

Output:

Sender Side Output

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-8$ python3.10 8_sender.py
Window size is considered as : 4
[=] UDP Server is starting at : ('127.0.0.1', 9090)
-----------------------------------------------
[=] Server waits for connection ...
[CONNECTED] Client ('127.0.0.1', 36253)
SENT : R-0
SENT : C-1
RECV : ['NACK', '0']
```

```
RECV : ['NACK', '12']
[Re-Tx] frame : 12
RECV : ['ACK', '13']
RECV : ['ACK', '12']
SENDER MESSAGE : RCCIIT ECE@SMN
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-8$
```

Receiver Side Output

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-8$ python3.10 8_receiver.py
Window size is considered as : 4
[CONNECTED] Server : ('127.0.0.1', 9090)
-----------------------------------------------
RECV : ['R', '0']
FRAME LOST
-----------------------------------------------
RECV : ['C', '1']
RECV OK
SENT : NACK-0
[=] Received buffer : ['', 'C']
SENT : ACK-1
```

```
-----------------------------------------------
RECV : ['M', '12']
RECV OK
[=] Received buffer : ['R', 'C', 'C', 'I', 'I', 'T', ' ', 'E', 'C', 'E', '@', 'S', 'M', 'N']
SENT : ACK-12
['R', 'C', 'C', 'I', 'I', 'T', ' ', 'E', 'C', 'E', '@', 'S', 'M', 'N']
RECEIVED MESSAGE : RCCIIT ECE@SMN
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-8$
```

Result

The Selective Repeat ARQ protocol was implemented successfully using UDP socket programming.

Conclusion

The experiment helped to understand frame retransmission, ACK/NACK handling, and sliding window operation in noisy channels.